Use Case 1

The vision

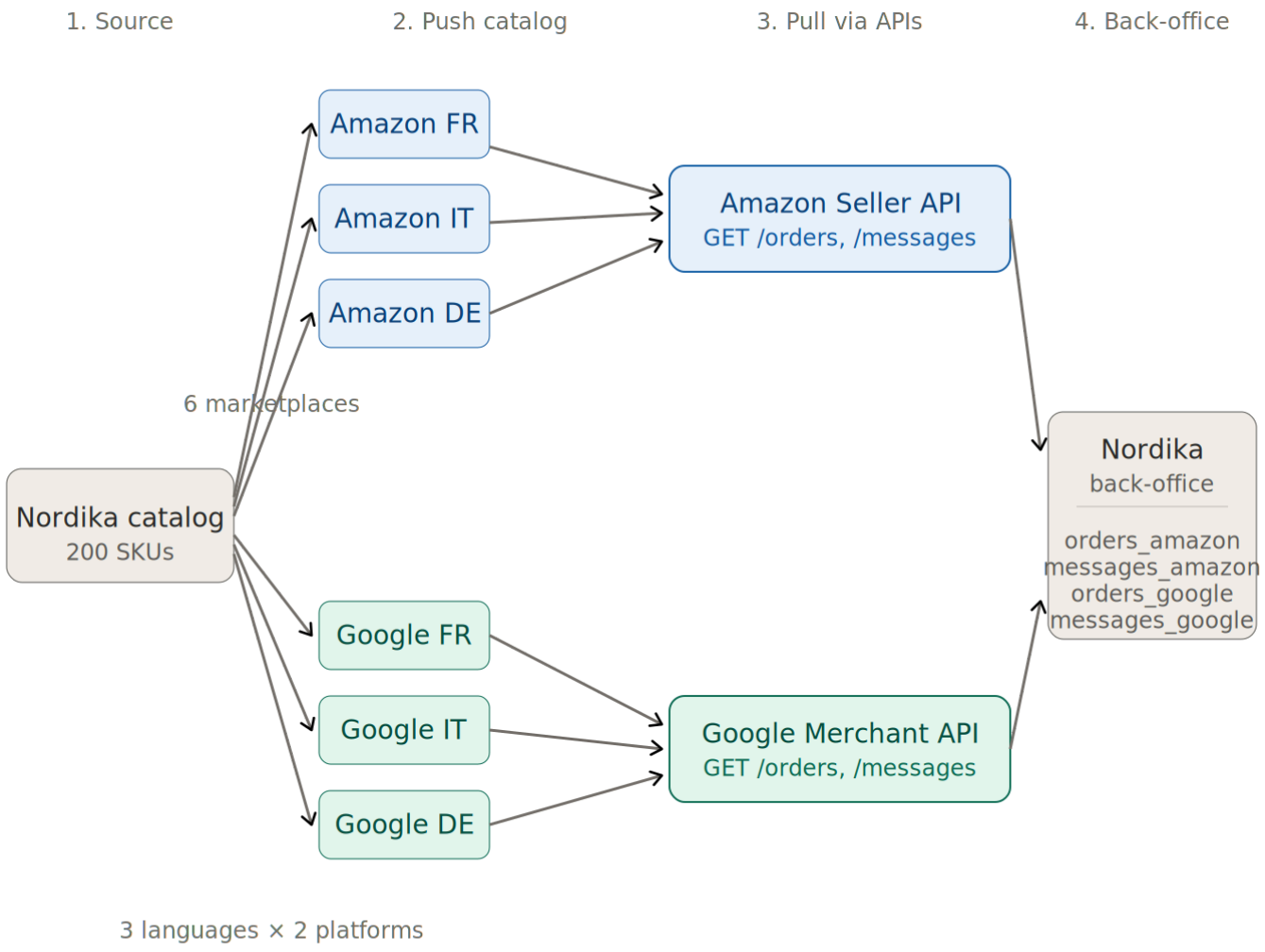
A single entrepreneur wants to launch a business. No website of their own, no physical store — they multiply reach by plugging directly into marketplaces and agentic channels. An **agentic system** handles the end-to-end operations autonomously: connection, catalog creation and enrichment, dynamic pricing, stock management, order processing and supply, and performance monitoring.

The problem

Running a marketplace seller business today normally requires a full team: someone for the catalog, someone for pricing, someone for stock and orders, someone for customer service. Agents can do better if they're designed with the seller in mind.

The case study: Nordika Studio

To keep the hackathon concrete, every team works on the same fictional seller: **Nordika Studio**, a French furniture brand run by a single founder, with a catalog of 200 SKUs across 6 marketplaces (Amazon FR/IT/DE and Google Shopping FR/IT/DE). A team would typically be needed to handle the workload — the hackathon explores how agents can shoulder that work alongside the founder.



Nordika pushes its catalog to 6 marketplaces (3 languages × 2 platforms), then pulls back orders and customer messages through two separate APIs: **Amazon Seller API** (for the 3 Amazon storefronts) and **Google Merchant API** (for the 3 Google storefronts). Each API returns data in its own format — unifying them is part of the work.

How the hackathon is structured

Every team starts with **catalog integration** — the common foundation. Before you can price, restock, or handle orders, you need your products live on the marketplaces. See `INTEGRATION.md` for the detailed brief.

Then each team picks **one** of three deep dives to build on top of their integration pipeline. The three deep-dive topics assume the catalog is already live on the 6 marketplaces — the order and message datasets you'll find in the kit reflect the activity flowing back once that's done. The guidelines below are deliberately light — the strategy is up to you.

Topic 1 — Dynamic pricing

Nordika's catalog is priced statically today. Some products don't sell because they're overpriced vs. the market; some categories cannibalize themselves with near-duplicate SKUs at different price points. Build an agent that surveys the market, detects outliers in Nordika's pricing, and proposes adjustments.

Things you might consider:

- Competitor prices — the catalog doesn't include them. Mock a competitor scraping pipeline if you need one.
- Sell-through signals — use `data/orders_amazon.jsonl` and `data/orders_google.jsonl` to identify fast movers, dormant stock, and everything in between.
- Strategy — discount-to-clear, category-aware repricing...
- Cadence — should prices update daily, weekly, event-driven?
- Guardrails — every pricing system needs safety rails. Where does the human stay in the loop?

Topic 2 — Order processing & supply

Nordika sells on 6 marketplaces with a single shared stock. Without coordination, they will regularly oversell: the same SKU gets sold on Amazon FR and Google DE on the same day, exceeding what's physically available. On top of that, stock eventually runs out and needs to be replenished. Build an agent that helps Nordika stay on top of this — the angle is up to you.

Things you might consider:

- Real data — orders come from two different marketplace APIs (`data/orders_amazon.jsonl`, `data/orders_google.jsonl`) with their own schemas. Unifying them is part of the work, and you'll probably notice things the seller hasn't noticed yet.
- Oversell & out-of-stock — the same unit getting sold twice, or a product that can't be fulfilled.
- Restocking — deciding when and how much to reorder.

Topic 3 — Customer service & messages

Nordika gets messages from buyers in three languages, across 6 marketplaces, in a mix of "where is my order?", "it arrived damaged", "can I return this?", and assembly-help questions. Most of these are repetitive enough to automate — but the tone, the accuracy, and the escalation rules matter a lot. Build an agent that handles the mundane and escalates the rest.

Things you might consider:

- Source material — messages come from two different marketplace APIs (`data/messages_amazon.jsonl`, `data/messages_google.jsonl`). They cover the full spectrum: late orders, damaged goods, return requests, angry customers, pre-sales questions. Thread structure is preserved when there was a back-and-forth with Nordika.
- Multilingual — FR, IT, DE responses that actually sound native.
- Tone & brand voice — a Nordika response shouldn't read like a chatbot.
- Escalation — which cases should always land on a human?

Ground rules — what you can invent

This is a hackathon brief, not a specification document. You are expected to mock anything that is not provided but needed for your solution to make sense — a fake internal product database, a mock

competitor price feed, a mock Amazon/Google API response, etc. If a realistic detail is missing, invent one that is plausible and move on.

The same applies to your solution itself: if your UI/UX, workflow, or agent needs a database, an authentication flow, a notification system, a logging backend, or any other piece of infrastructure, you can mock it.

What matters

Whatever form your solution takes (web app, conversational agent, orchestrated agents, fine-tuned model, hybrid...), it should demonstrate:

- How a non-technical seller triggers the workflow and stays in control.
- How progress, errors, and decisions are surfaced to the seller.
- Bonus: how the solution would scale to 10,000 SKUs instead of 200.

Data you get

File	Description
`data/supplier_catalog_nordika_200.csv`	Nordika's source catalog: 200 products, 80 columns, bilingual (EN/FR), intentionally imperfect
`schemas/amazon_common_attributes.yaml`	Amazon feed attributes that apply to every product type
`schemas/amazon_product_type_attributes.yaml`	Amazon product-type-specific attributes and overrides
`schemas/amazon_product_type.csv`	A selection of Amazon `product_type` values (~250 entries)
`schemas/google_shopping_attributes.yaml`	Google Shopping feed attributes and constraints
`schemas/google_taxonomy_fr.txt`, `schemas/google_taxonomy_it.txt`, `schemas/google_taxonomy_de.txt`	Google Shopping category taxonomies, one per language
`data/orders_amazon.jsonl`	Orders from the 3 Amazon storefronts (FR/IT/DE) — Amazon Seller API format
`data/orders_google.jsonl`	Orders from the 3 Google storefronts (FR/IT/DE) — Google Merchant API format
`data/messages_amazon.jsonl`	Customer messages from the 3 Amazon storefronts —

File	Description
	thread-grouped
`data/messages_google.jsonl`	Customer messages from the 3 Google storefronts — thread-grouped

Next steps

- Read `INTEGRATION.md` for the detailed integration brief (common to all teams).
- Pick your deep-dive topic.
- Start building.